

Interface 구조를 언제, 어떻게 사용해야 하는가

영웅호걸 3차 코드리뷰
- 윤지원 -

Interface란?

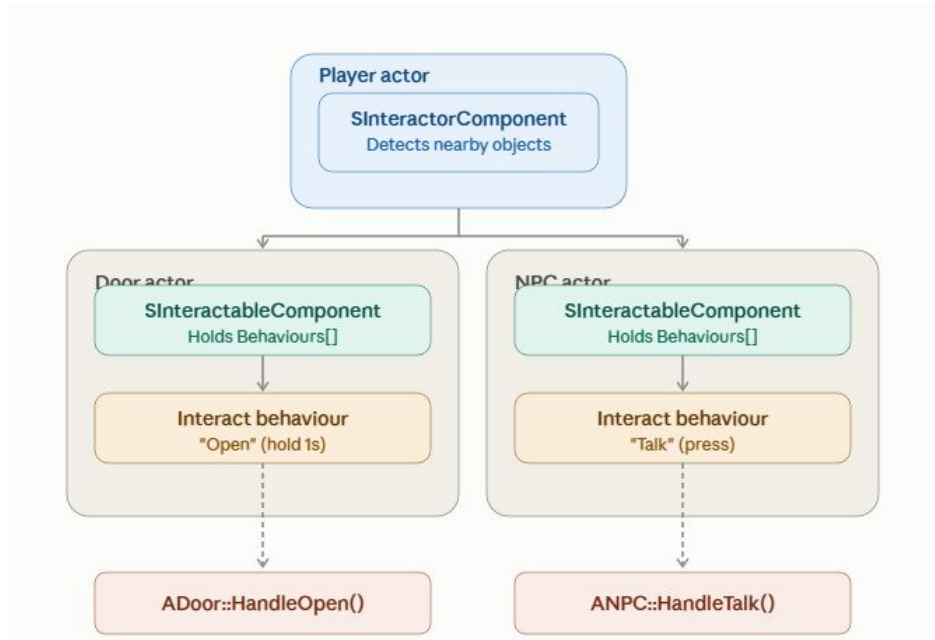
- 상태와 구현 없이 “행동”만 선언하는 구조
- 실제 동작은 이를 구현하는 클래스가 채운다.

장점

- 구체 클래스를 몰라도 기능을 사용할 수 있으므로, **의존성을 줄일 수 있다.**
- 함수의 호출을 **규격화**하여 전달할 수 있다.
- **interface** 구조가 바뀌지 않으면 내부를 수정해도 외부에 영향을 주지 않으므로 **유지보수에 용이**하다.
- 같은 함수를 호출하되, 동작하는 행동이 다를 때 **확장하기 좋다.**

과거 상호작용 시스템 구조

- 의존성 문제 => 델리게이트로 해결
- 규격화, 유지보수 => public 함수를 잘 구성
- 확장성 => 델리게이트로 서로 다른 동작은 이벤트로 등록함..!



이런 상황에서는 **Interface**를 쓰면 구조만 더 복잡해지는게 아닐까?
상속받는 인터페이스가 너무 많으면 그것대로 코드를 파악하기 힘들어지는게 아닐까?

결론

상호작용 시스템은 서로 다른 클래스(Door, Item, NPC 등) 사이에 공통된 행동(interaction)을 정의해야 하기 때문에 **Interface**를 주로 사용하게 된다.

이런 공통적인 기능을 서로 다른 클래스에 정의하고 싶을 때, **Interface**를 사용하면 코드 가독성도 좋고 하나의 함수로 묶어서 관리할 수 있다.

이번 상호작용 시스템은 **Interface**도 넣어서 만들어보겠습니다.